

Task 1: TO Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

```
vedadripinninti@MacbookAir: ~/s + v
vedadripinninti@MacbookAir:~$ mkdir skill_week8
vedadripinninti@MacbookAir:~$ cd skill_week8
vedadripinninti@MacbookAir:~/skill_week8$ nano 1.c
```

CODE:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd = open("combined.txt",
                  O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0) {
        perror("open");
        return 1;
    }

    int old_out = dup(STDOUT_FILENO);
    int old_err = dup(STDERR_FILENO);

    /* Redirect stdout and stderr to same file */
    dup2(fd, STDOUT_FILENO);
    dup2(fd, STDERR_FILENO);
    close(fd);

    printf("Normal output: Program started\n");
    fflush(stdout);

    fprintf(stderr, "Error output: File not found\n");
    fflush(stderr);

    printf("Normal output: Program finished\n");
    fflush(stdout);

    /* Restore streams */
    dup2(old_out, STDOUT_FILENO);
    dup2(old_err, STDERR_FILENO);

    close(old_out);
    close(old_err);

    printf("Streams restored successfully.\n");

    return 0;
}
```

OUTPUT:

```
vedadripinninti@MacbookAir: ~/s + v
vedadripinninti@MacbookAir:~$ mkdir skill_week8
vedadripinninti@MacbookAir:~$ cd skill_week8
vedadripinninti@MacbookAir:~/skill_week8$ nano 1.c
vedadripinninti@MacbookAir:~/skill_week8$ gcc 1.c -o 1
vedadripinninti@MacbookAir:~/skill_week8$ ./1
Streams restored successfully.
vedadripinninti@MacbookAir:~/skill_week8$
```

Task 2: To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.**CODE:**

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char cmd[200];
    char *left, *right;
    char *a[10], *b[10];
    int i = 0, j = 0, pipefd[2];

    printf("Enter command: ");
    fgets(cmd, sizeof(cmd), stdin);
    cmd[strcspn(cmd, "\n")] = 0;

    left = strtok(cmd, "|");
    right = strtok(NULL, "|");

    if (!left || !right) {
        printf("Invalid syntax: use command1 | command2\n");
        return 1;
    }

    a[i] = strtok(left, " ");
    while (a[i] && i < 9)
        a[++i] = strtok(NULL, " ");

    b[j] = strtok(right, " ");
    while (b[j] && j < 9)
        b[++j] = strtok(NULL, " ");

    /* Simple execution-plan display */
    printf("\nExecution Plan:\n");
    printf("Command 1: %s\n", a[0]);
    printf("Command 2: %s\n", b[0]);
    printf("Pipe: stdout of Command 1 -> stdin of Command 2\n");

    pipe(pipefd);

    if (fork() == 0) {
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
        execvp(a[0], a);
        perror("exec");
        exit(1);
    }

    if (fork() == 0) {
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
        execvp(b[0], b);
        perror("exec");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    wait(NULL);
    wait(NULL);

    return 0;
}

```

OUTPUT:

```

vedadripinninti@MacbookAir:~/skill_week8$
vedadripinninti@MacbookAir:~/skill_week8$ nano 2.c
vedadripinninti@MacbookAir:~/skill_week8$ gcc 2.c -o 2
vedadripinninti@MacbookAir:~/skill_week8$ ./2
Enter command: pwd
Invalid syntax: use command1 | command2
vedadripinninti@MacbookAir:~/skill_week8$ ./2
Enter command: hello server
Invalid syntax: use command1 | command2

```

SESSION-16

Task 1: To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Starting background job...\n");

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* Background task */
        printf("Background job running...\n");
        sleep(5);
        printf("Background job finished.\n");
        exit(0);
    }

    /* Parent does not wait */
    printf("Job started in background. PID = %d\n", pid);
    printf("Prompt returned immediately.\n");

    /* Non-blocking check */
    sleep(1);

    if (waitpid(pid, NULL, WNOHANG) == 0)
        printf("Job is still running.\n");
    else
        printf("Job completed.\n");

    /* Wait only to clean up the child */
    waitpid(pid, NULL, 0);

    printf("Background job monitored successfully.\n");

    return 0;
}
```

OUTPUT:

```
vedadripinninti@MacbookAir:~/skill_week8$
vedadripinninti@MacbookAir:~/skill_week8$ nano 3.c
vedadripinninti@MacbookAir:~/skill_week8$ gcc 3.c -o 3
vedadripinninti@MacbookAir:~/skill_week8$ ./3
Starting background job...
Job started in background. PID = 715
Prompt returned immediately.
Background job running...
Job is still running.
Background job finished.
Background job monitored successfully.
vedadripinninti@MacbookAir:~/skill_week8$
```

Task 2: To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

```

#include <stdlib.h>

struct Job {
    int id;
    pid_t pid;
    pid_t pgid;
    char state[20];
};

int main()
{
    struct Job jobs[5];
    int count = 0;
    pid_t pid;

    /* Create background job */
    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        sleep(3);
        exit(0);
    }

    /* Store job information */
    jobs[count].id = 1;
    jobs[count].pid = pid;
    jobs[count].pgid = getpgid(pid);
    sprintf(jobs[count].state, "Running");
    count++;

    printf("\nJob Table:\n");
    printf("ID\tPID\tPGID\tState\n");
    printf("%d\t%d\t%d\t%s\n",
           jobs[0].id, jobs[0].pid,
           jobs[0].pgid, jobs[0].state);

    /* Check job status */
    sleep(1);

    if (waitpid(pid, NULL, WNOHANG) == 0) {
        printf("\nJob %d is still running.\n", jobs[0].id);
    }

    /* Wait for completion */
    waitpid(pid, NULL, 0);

    sprintf(jobs[0].state, "Completed");

    printf("Job %d completed.\n", jobs[0].id);

    /* Remove completed job */
    count = 0;

    printf("\nJob Table after removing completed jobs:\n");
    printf("Number of active jobs: %d\n", count);

    return 0;
}

```

OUTPUT:

```

vedadripinninti@MacbookAir:~/skill_week8$
vedadripinninti@MacbookAir:~/skill_week8$ nano 4.c
vedadripinninti@MacbookAir:~/skill_week8$ gcc 4.c -o 4
vedadripinninti@MacbookAir:~/skill_week8$ ./4

```

```

Job Table:
ID      PID      PGID      State
1        791        790      Running

```

```

Job 1 is still running.
Job 1 completed.

```

```
Job Table after removing completed jobs:  
Number of active jobs: 0  
vedadripinninti@MacbookAir:~/skill_week8$
```